

Metabolomic Data Analysis with MetaboAnalyst 6.0

Name: guest3216449863702285686

April 29, 2026

1 Data Processing and Normalization

1.1 Reading and Processing the Raw Data

MetaboAnalyst accepts a variety of data types generated in metabolomic studies, including compound concentration data, binned NMR/MS spectra data, NMR/MS peak list data, as well as MS spectra (NetCDF, mzXML, mzDATA). Users need to specify the data types when uploading their data in order for MetaboAnalyst to select the correct algorithm to process them. Table 1 summarizes the result of the data processing steps.

1.1.1 Reading Binned Spectral Data

The binned spectra data should be uploaded in comma separated values (.csv) format. Samples can be in rows or columns, with class labels immediately following the sample IDs.

Samples are in rows and features in columns The uploaded file is in comma separated values (.csv) format. The uploaded data file contains 50 (samples) by 200 (spectra bins) data matrix.

1.1.2 Data Integrity Check

Before data analysis, a data integrity check is performed to make sure that all the necessary information has been collected. The class labels must be present and contain only two classes. If samples are paired, the class label must be from $-n/2$ to -1 for one group, and 1 to $n/2$ for the other group (n is the sample number and must be an even number). Class labels with same absolute value are assumed to be pairs. Compound concentration or peak intensity values should all be non-negative numbers. By default, all missing values, zeros and negative values will be replaced by the half of the minimum positive value found within the data (see next section)

1.1.3 Missing value imputations

Too many zeroes or missing values will cause difficulties for downstream analysis. MetaboAnalyst offers several different methods for this purpose. The default method replaces all the missing and zero values with a small values (the half of the minimum positive values in the original data) assuming to be the detection limit. The assumption of this approach is that most missing values are caused by low abundance metabolites (i.e. below the detection limit). In addition, since zero values may cause problem for data normalization (i.e. log), they are also replaced with this small value. User can also specify other methods, such as replace by mean/median, or use K-Nearest Neighbours (KNN), Probabilistic PCA (PPCA), Bayesian PCA (BPCA) method, Singular Value Decomposition (SVD) method to impute the missing values ¹. Please choose the one that is the most appropriate for your data.

¹Stacklies W, Redestig H, Scholz M, Walther D, Selbig J. *pcaMethods: a bioconductor package, providing PCA methods for incomplete data.*, Bioinformatics 2007 23(9):1164-1167

1.1.4 Data Filtering

The purpose of the data filtering is to identify and remove variables that are unlikely to be of use when modeling the data. No phenotype information are used in the filtering process, so the result can be used with any downstream analysis. This step can usually improves the results. Data filter is strongly recommended for datasets with large number of variables (> 250) datasets contain much noise (i.e.chemometrics data). Filtering can usually improve your results².

*For data with number of variables < 250 , this step will reduce 5% of variables; For variable number between 250 and 500, 10% of variables will be removed; For variable number bwtween 500 and 1000, 25% of variables will be removed; And 40% of variabled will be removed for data with over 1000 variables. The None option is only for less than 5000 features. Over that, if you choose None, the IQR filter will still be applied. In addition, the maximum allowed number of variables is **10000***

Feature filtering based on ““ Interquantile Range ““ - removed $< b > 0$ based on the cutoff.

Table 1: Summary of data processing results

	Features (positive)	Missing/Zero	Features (processed)
C002	194	6	200
C004	189	11	200
C005	191	9	200
C006	195	5	200
C007	200	0	200
C009	186	14	200
C010	196	4	200
C011	177	23	200
C012	189	11	200
C015	188	12	200
C016	188	12	200
C017	198	2	200
C019	181	19	200
C020	184	16	200
C021	187	13	200
C022	191	9	200
C024	190	10	200
C026	195	5	200
C028	196	4	200
C029	192	8	200
C030	182	18	200
C031	179	21	200
C032	191	9	200
C033	189	11	200
C034	199	1	200
P002	195	5	200
P012	187	13	200
P014	200	0	200
P027	200	0	200
P034	198	2	200
P037	187	13	200
P038	195	5	200
P041	178	22	200
P042	198	2	200
P049	189	11	200
P056	190	10	200
P058	179	21	200
P060	190	10	200
P064	200	0	200
P065	198	2	200
P070	190	10	200
P080	196	4	200
P085	200	0	200
P086	193	7	200
P089	199	1	200
P092	191	9	200
P099	190	10	200
P113	152	48	200
P013b	191	9	200
P100b	199	1	200

²Hackstadt AJ, Hess AM. *Filtering for increased power for microarray data analysis*, BMC Bioinformatics. 2009; 10: 11.

1.2 Data Normalization

The data is stored as a table with one sample per row and one variable (bin/peak/metabolite) per column. The normalization procedures implemented below are grouped into four categories. Sample specific normalization allows users to manually adjust concentrations based on biological inputs (i.e. volume, mass); row-wise normalization allows general-purpose adjustment for differences among samples; data transformation and scaling are two different approaches to make features more comparable. You can use one or combine both to achieve better results.

The normalization consists of the following options:

1. Row-wise procedures:
 - Sample specific normalization (i.e. normalize by dry weight, volume)
 - Normalization by the sum
 - Normalization by the sample median
 - Normalization by a reference sample (probabilistic quotient normalization)³
 - Normalization by a pooled or average sample from a particular group
 - Normalization by a reference feature (i.e. creatinine, internal control)
 - Quantile normalization
2. Data transformation :
 - Log transformation (base 10)
 - Square root transformation
 - Cube root transformation
3. Data scaling:
 - Mean centering (mean-centered only)
 - Auto scaling (mean-centered and divided by standard deviation of each variable)
 - Pareto scaling (mean-centered and divided by the square root of standard deviation of each variable)
 - Range scaling (mean-centered and divided by the value range of each variable)

Figure 1 shows the effects before and after normalization.

Row-wise normalization: Normalization to constant sum; Data transformation: Log10 Normalization; Data scaling: Pareto Scaling.

³Dieterle F, Ross A, Schlotterbeck G, Senn H. *Probabilistic quotient normalization as robust method to account for dilution of complex biological mixtures. Application in 1H NMR metabonomics*, 2006, Anal Chem 78 (13);4281 - 4290

Density

Before Normalization After Normalization

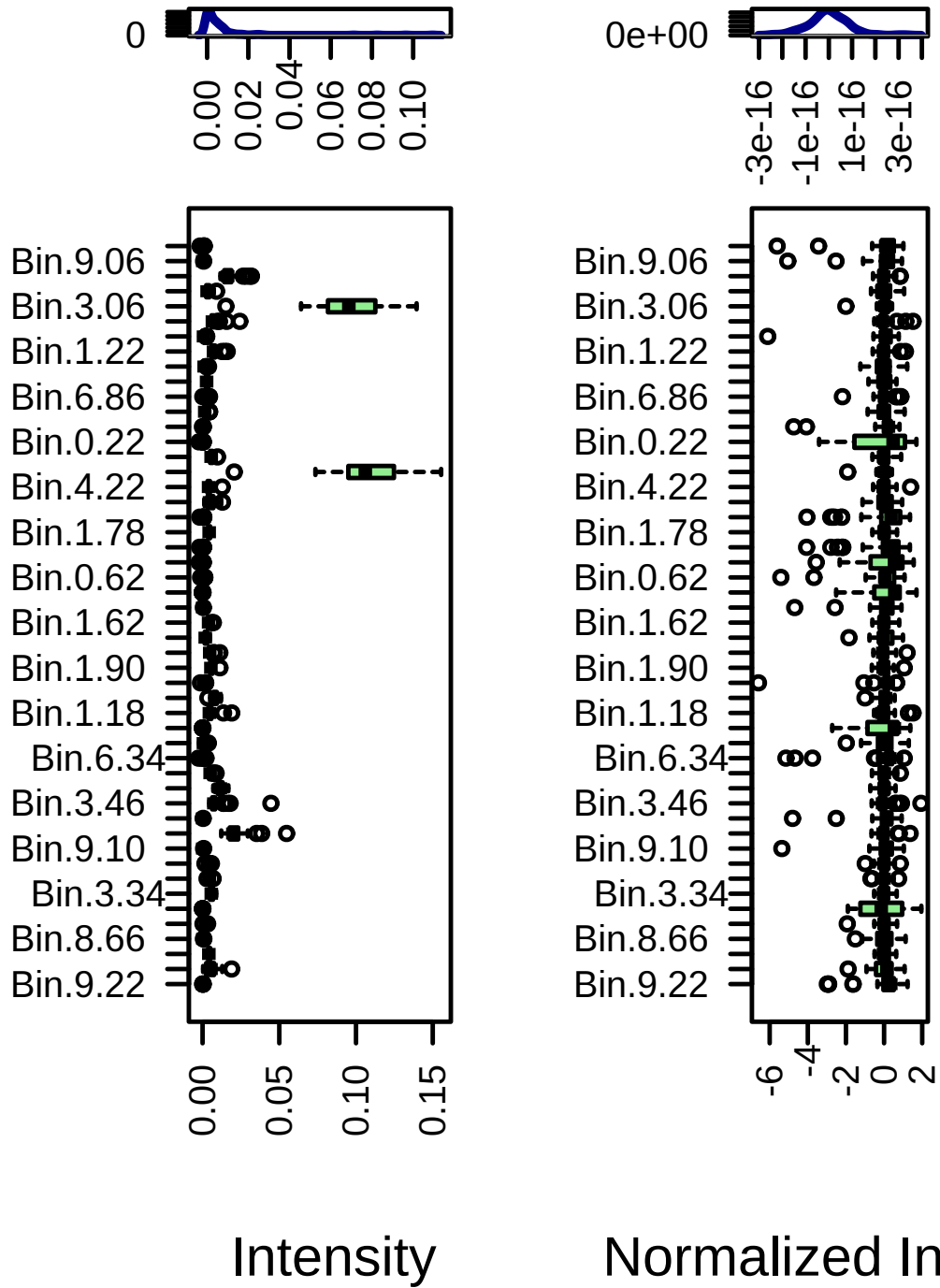


Figure 1: Box plots and kernel density plots before and after normalization. The boxplots show at most 50 features due to space limit. The density plots are based on all samples.

2 Statistical and Machine Learning Data Analysis

MetaboAnalyst offers a variety of methods commonly used in metabolomic data analyses. They include:

1. Univariate analysis methods:
 - Fold Change Analysis
 - T-tests
 - Volcano Plot
 - One-way ANOVA and post-hoc analysis
 - Correlation analysis
2. Multivariate analysis methods:
 - Principal Component Analysis (PCA)
 - Partial Least Squares - Discriminant Analysis (PLS-DA)
3. Robust Feature Selection Methods in microarray studies
 - Significance Analysis of Microarray (SAM)
 - Empirical Bayesian Analysis of Microarray (EBAM)
4. Clustering Analysis
 - Hierarchical Clustering
 - Dendrogram
 - Heatmap
 - Partitional Clustering
 - K-means Clustering
 - Self-Organizing Map (SOM)
5. Supervised Classification and Feature Selection methods
 - Random Forest
 - Support Vector Machine (SVM)

Please note: some advanced methods are available only for two-group sample analysis.

2.1 Univariate Analysis

Univariate analysis methods are the most common methods used for exploratory data analysis. For two-group data, MetaboAnalyst provides Fold Change (FC) analysis, t-tests, and volcano plot which is a combination of the first two methods. All three these methods support both unpaired and paired analyses. For multi-group analysis, MetaboAnalyst provides two types of analysis - one-way analysis of variance (ANOVA) with associated post-hoc analyses, and correlation analysis to identify significant compounds that follow a given pattern. The univariate analyses provide a preliminary overview about features that are potentially significant in discriminating the conditions under study.

For paired fold change analysis, the algorithm first counts the total number of pairs with fold changes that are consistently above/below the specified FC threshold for each variable. A variable will be reported as significant if this number is above a given count threshold (default $> 75\%$ of pairs/variable)

Figure 2 shows the important features identified by t-tests. Table 2 shows the details of these features;

Please note, the purpose of fold change is to compare absolute value changes between two group means. Therefore, the data before column normalization will be used instead. Also note, the result is plotted in log2 scale, so that same fold change (up/down regulated) will have the same distance to the zero baseline.

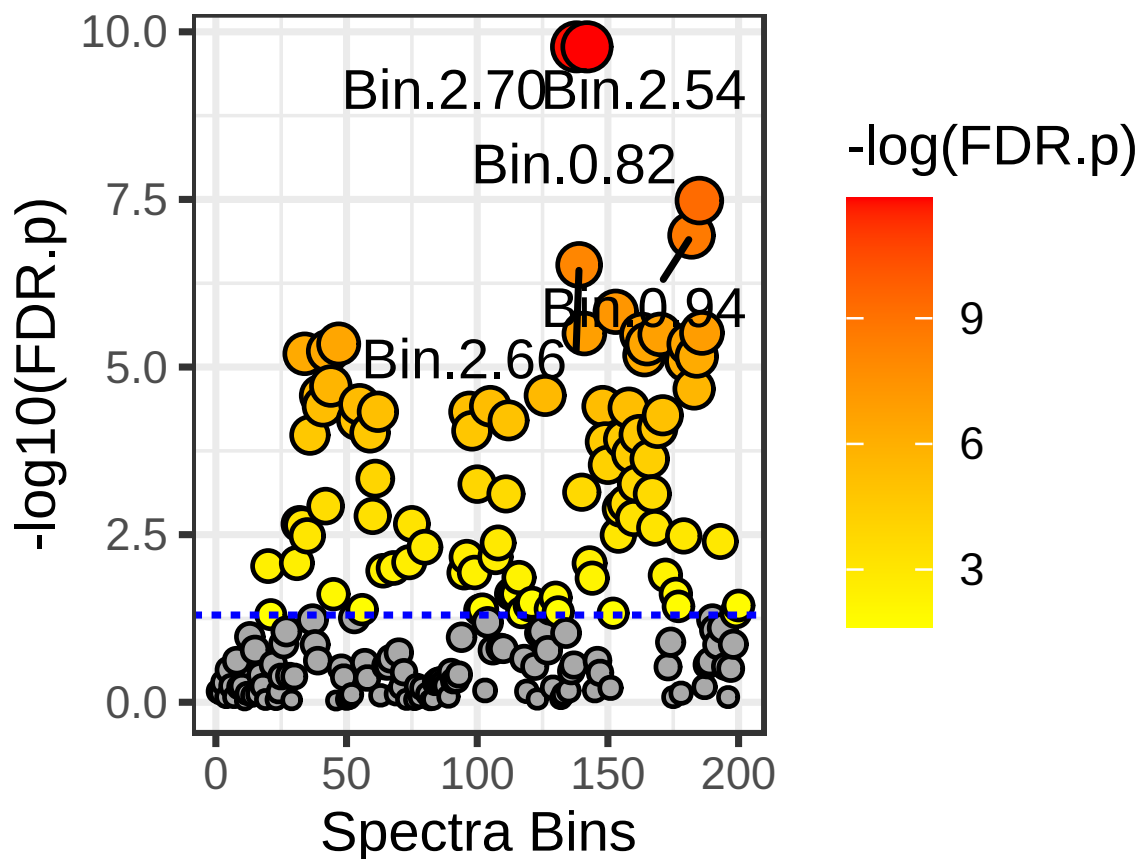


Figure 2: Important features selected by t-tests with threshold 0.05. The red circles represent features above the threshold. Note the p values are transformed by $-\log_{10}$ so that the more significant features (with smaller p values) will be plotted higher on the graph.

Table 2: Top 50 features identified by t-tests

	Spectra Bins	t.stat	p.value	-log10(p)	FDR
1	Bin.2.54	9.9238	1.371e-12	11.863	1.6753e-10
2	Bin.2.70	9.9365	1.6753e-12	11.776	1.6753e-10
3	Bin.0.82	-8.7471	4.9338e-10	9.3068	3.2892e-08
4	Bin.0.94	-8.6461	2.1742e-09	8.6627	1.0871e-07
5	Bin.2.66	7.0928	7.4928e-09	8.1254	2.9971e-07
6	Bin.2.10	-6.9467	4.4978e-08	7.347	1.4993e-06
7	Bin.0.78	-6.8538	1.0844e-07	6.9648	3.0982e-06
8	Bin.2.58	6.2245	1.2647e-07	6.898	3.1617e-06
9	Bin.1.42	-6.4396	1.4757e-07	6.831	3.2568e-06
10	Bin.1.70	-6.3772	1.6284e-07	6.7882	3.2568e-06
11	Bin.0.98	-6.8114	2.8652e-07	6.5429	4.5225e-06
12	Bin.8.14	-5.964	2.9193e-07	6.5347	4.5225e-06
13	Bin.1.62	-6.0123	2.9396e-07	6.5317	4.5225e-06
14	Bin.8.30	-6.1741	4.0504e-07	6.3925	5.7863e-06
15	Bin.8.66	-5.8175	4.7586e-07	6.3225	6.3448e-06
16	Bin.1.66	-5.8845	5.3155e-07	6.2745	6.6444e-06
17	Bin.0.86	-6.1568	5.9148e-07	6.2281	6.9586e-06
18	Bin.1.02	-6.1814	7.2845e-07	6.1376	8.0939e-06
19	Bin.8.26	-5.5099	1.8233e-06	5.7392	1.9192e-05
20	Bin.0.90	-5.7704	2.1208e-06	5.6735	2.1208e-05
21	Bin.8.42	-5.4102	2.8108e-06	5.5512	2.654e-05
22	Bin.3.14	5.479	2.9194e-06	5.5347	2.654e-05
23	Bin.7.82	5.2899	4.1457e-06	5.3824	3.6049e-05
24	Bin.2.30	-5.1946	4.7796e-06	5.3206	3.8406e-05
25	Bin.3.98	5.1626	4.8299e-06	5.3161	3.8406e-05
26	Bin.8.38	-5.2104	4.9928e-06	5.3017	3.8406e-05
27	Bin.1.90	-5.1248	5.4696e-06	5.262	4.0516e-05
28	Bin.7.54	5.1891	6.597e-06	5.1807	4.6476e-05
29	Bin.4.26	-5.071	6.739e-06	5.1714	4.6476e-05
30	Bin.1.38	-5.1248	7.8596e-06	5.1046	5.2397e-05
31	Bin.7.86	5.0442	9.5028e-06	5.0221	6.1308e-05
32	Bin.3.70	-5.0864	9.9986e-06	5.0001	6.2491e-05
33	Bin.1.46	-4.9405	1.3448e-05	4.8714	8.1501e-05
34	Bin.4.22	-4.896	1.517e-05	4.819	8.9232e-05
35	Bin.7.66	4.8892	1.675e-05	4.776	9.5715e-05
36	Bin.1.74	-4.8248	1.835e-05	4.7364	0.00010195
37	Bin.8.58	-4.7458	1.9155e-05	4.7177	0.00010354
38	Bin.1.98	-4.7834	2.2722e-05	4.6436	0.00011959
39	Bin.2.26	-4.6598	2.5423e-05	4.5948	0.00013038
40	Bin.1.86	-4.6318	3.953e-05	4.4031	0.00019765
41	Bin.1.58	-4.5099	4.807e-05	4.3181	0.00023449
42	Bin.2.22	-4.3981	6.0419e-05	4.2188	0.00028771
43	Bin.7.58	4.3317	9.9193e-05	4.0035	0.00046136
44	Bin.1.78	-4.2176	0.00012151	3.9154	0.0005523
45	Bin.4.14	-4.181	0.00012552	3.9013	0.00055787
46	Bin.2.62	4.0988	0.00016939	3.7711	0.00073646
47	Bin.3.74	-4.177	0.0001851	3.7326	0.00078071
48	Bin.1.54	-4.1615	0.00018737	3.7273	0.00078071
49	Bin.1.94	-3.9361	0.00026685	3.5737	0.0010892
50	Bin.8.34	-3.9201	0.00029675	3.5276	0.001187

2.2 Principal Component Analysis (PCA)

PCA is an unsupervised method aiming to find the directions that best explain the variance in a data set (X) without referring to class labels (Y). The data are summarized into much fewer variables called *scores* which are weighted average of the original variables. The weighting profiles are called *loadings*. The PCA analysis is performed using the `prcomp` package. The calculation is based on singular value decomposition.

The Rscript `chemometrics.R` is required. Figure 3 is pairwise score plots providing an overview of the various separation patterns among the most significant PCs; Figure 4 is the scree plot showing the variances explained by the selected PCs; Figure 5 shows the 2-D scores plot between selected PCs; Figure 6 shows the biplot between the selected PCs. Interactive 3-D scores plots are not included here and can be directly downloaded from website.

Figure 3: Pairwise score plots between the selected PCs. The explained variance of each PC is shown in the corresponding diagonal cell.

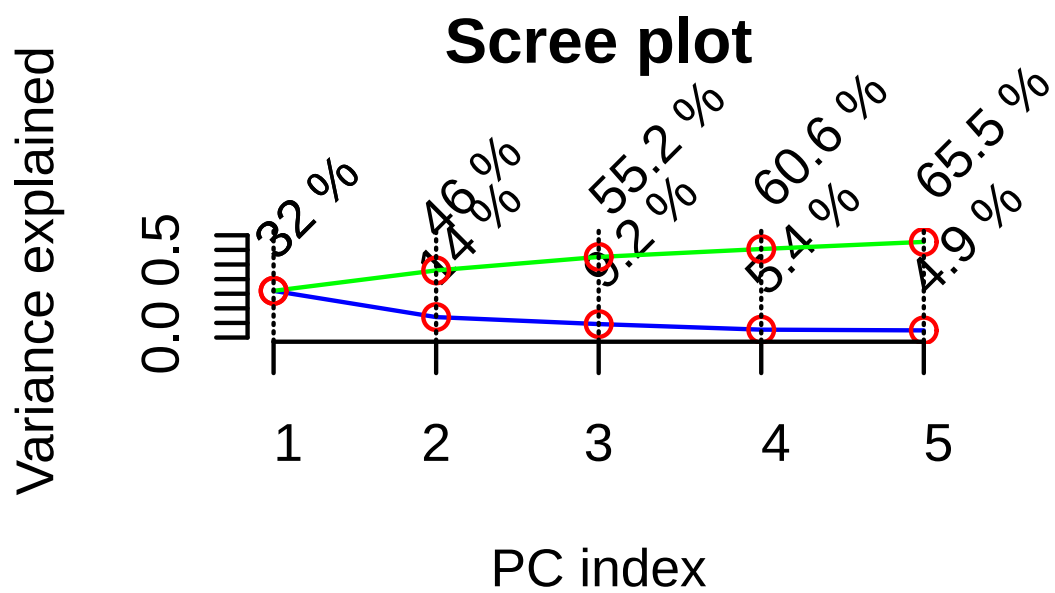


Figure 4: Scree plot shows the variance explained by PCs. The green line on top shows the accumulated variance explained; the blue line underneath shows the variance explained by individual PC.

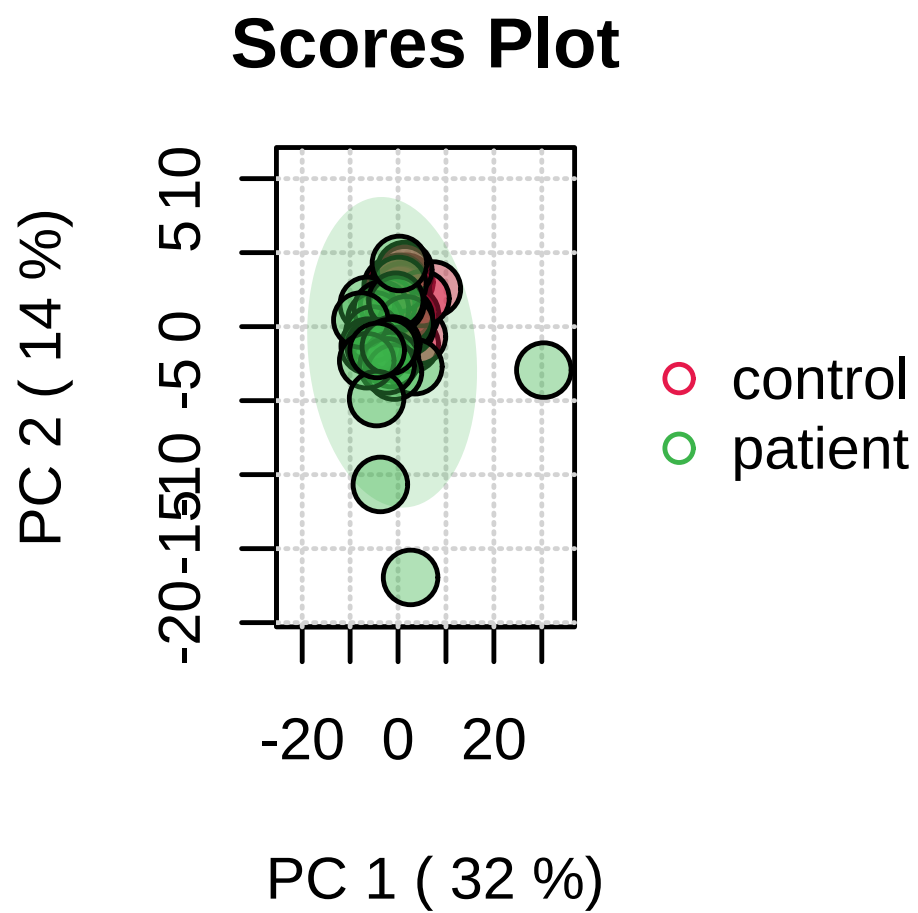


Figure 5: Scores plot between the selected PCs. The explained variances are shown in brackets.

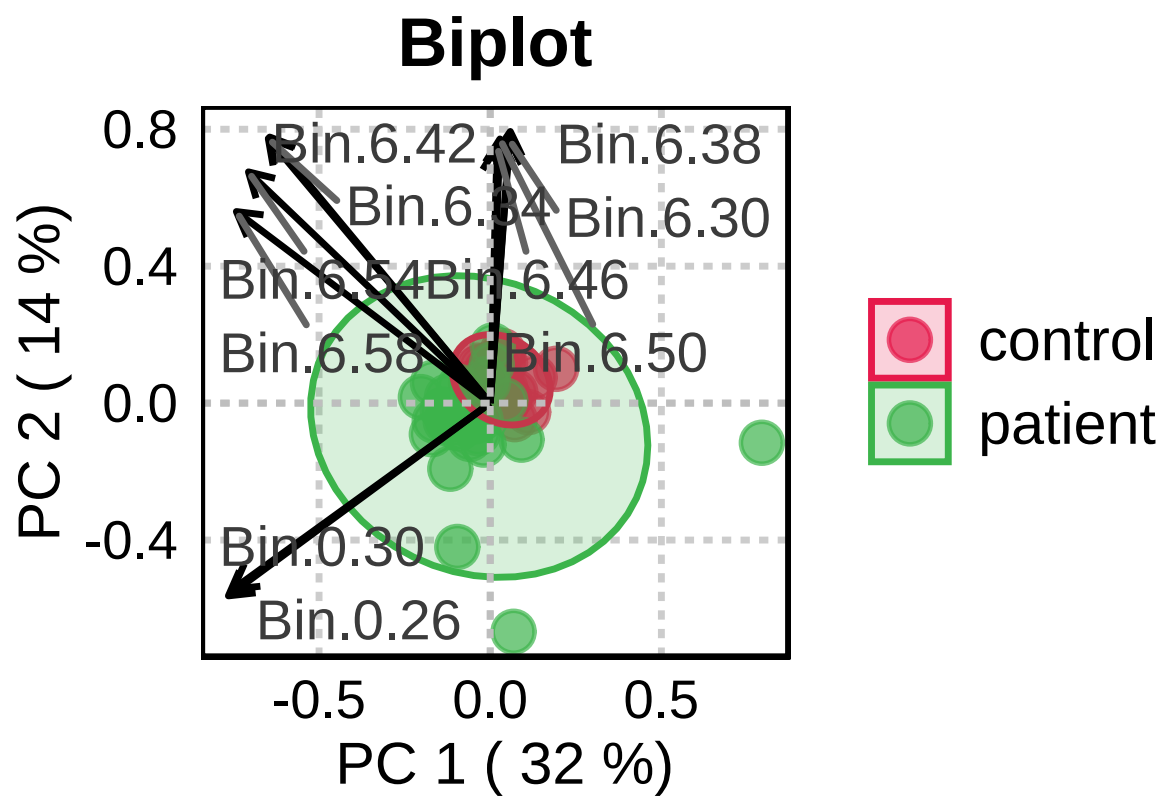


Figure 6: PCA biplot between the selected PCs. Note, you may want to test different centering and scaling normalization methods for the biplot to be displayed properly.

2.3 Partial Least Squares - Discriminant Analysis (PLS-DA)

PLS is a supervised method that uses multivariate regression techniques to extract via linear combination of original variables (X) the information that can predict the class membership (Y). The PLS regression is performed using the `pls` function provided by R `pls` package⁴. The classification and cross-validation are performed using the corresponding wrapper function offered by the `caret` package⁵.

To assess the significance of class discrimination, a permutation test was performed. In each permutation, a PLS-DA model was built between the data (X) and the permuted class labels (Y) using the optimal number of components determined by cross validation for the model based on the original class assignment. MetaboAnalyst supports two types of test statistics for measuring the class discrimination. The first one is based on prediction accuracy during training. The second one is separation distance based on the ratio of the between group sum of the squares and the within group sum of squares (B/W-ratio). If the observed test statistic is part of the distribution based on the permuted class assignments, the class discrimination cannot be considered significant from a statistical point of view.⁶

There are two variable importance measures in PLS-DA. The first, Variable Importance in Projection (VIP) is a weighted sum of squares of the PLS loadings taking into account the amount of explained Y-variation in each dimension. Please note, VIP scores are calculated for each components. When more than components are used to calculate the feature importance, the average of the VIP scores are used. The other importance measure is based on the weighted sum of PLS-regression. The weights are a function of the reduction of the sums of squares across the number of PLS components. Please note, for multiple-group (more than two) analysis, the same number of predictors will be built for each group. Therefore, the coefficient of each feature will be different depending on which group you want to predict. The average of the feature coefficients are used to indicate the overall coefficient-based importance.

Figure 7 shows the overview of scores plots; Figure 8 shows the 2-D scores plot between selected components; Figure 9 shows the 3-D scores plot between selected components; Figure 10 shows the loading plot between the selected components; Figure 11 shows the classification performance with different number of components; Figure 12 shows the results of permutation test for model validation; Figure 13 shows important features identified by PLS-DA.

⁴Ron Wehrens and Bjorn-Helge Mevik. *pls: Partial Least Squares Regression (PLSR) and Principal Component Regression (PCR)*, 2007, R package version 2.1-0

⁵Max Kuhn. Contributions from Jed Wing and Steve Weston and Andre Williams. *caret: Classification and Regression Training*, 2008, R package version 3.45

⁶Bijlsma et al. *Large-Scale Human Metabolomics Studies: A Strategy for Data (Pre-) Processing and Validation*, Anal Chem. 2006, 78 567 - 574

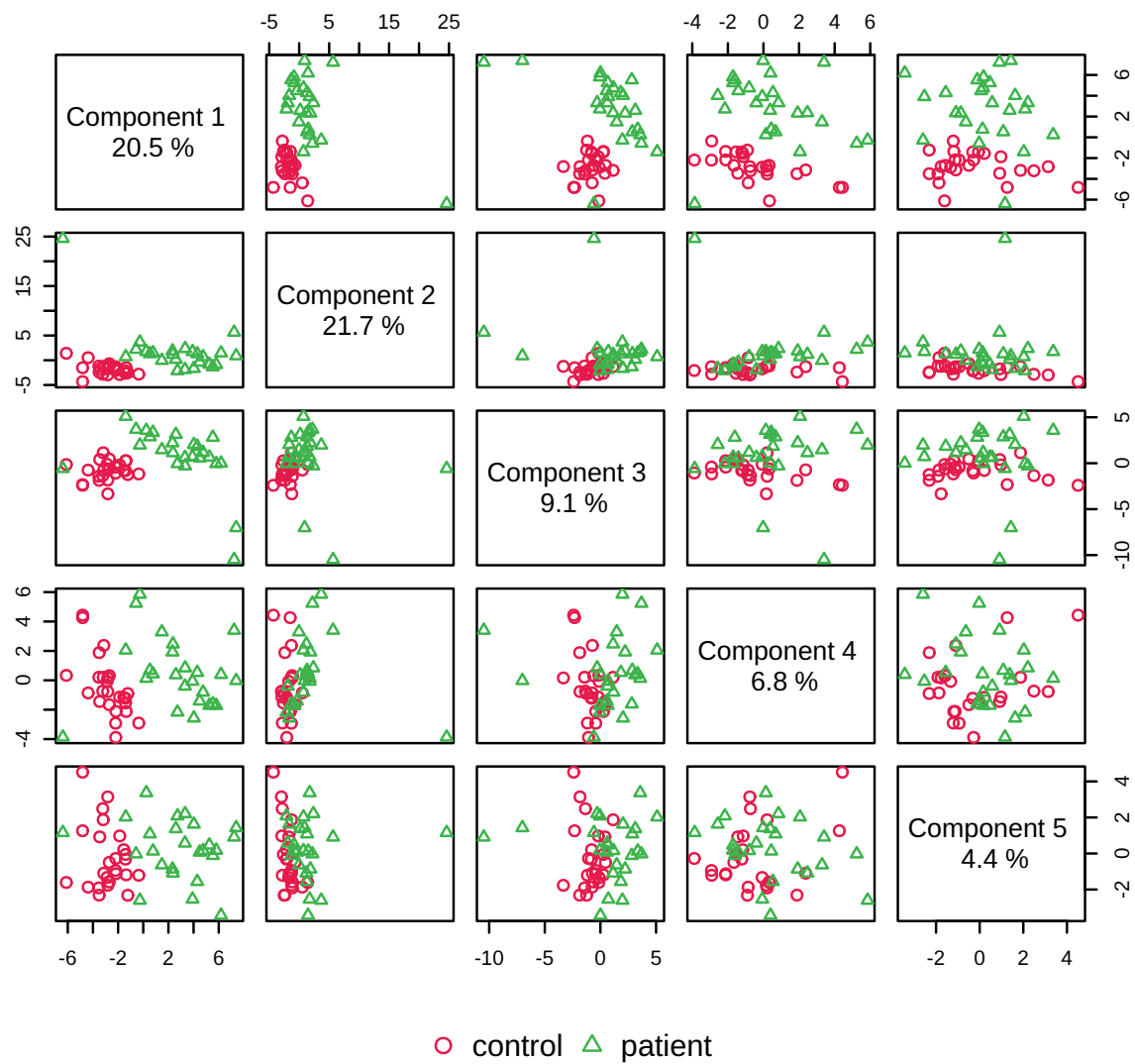


Figure 7: Pairwise scores plots between the selected components. The explained variance of each component is shown in the corresponding diagonal cell.

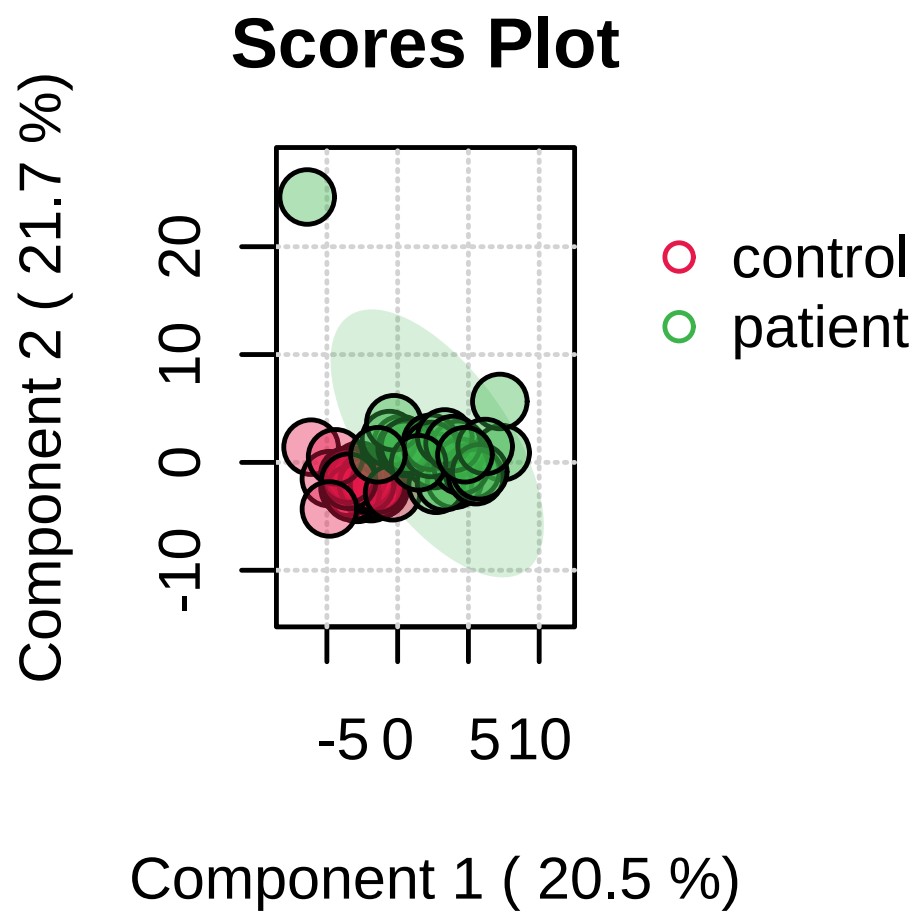


Figure 8: Scores plot between the selected PCs. The explained variances are shown in brackets.

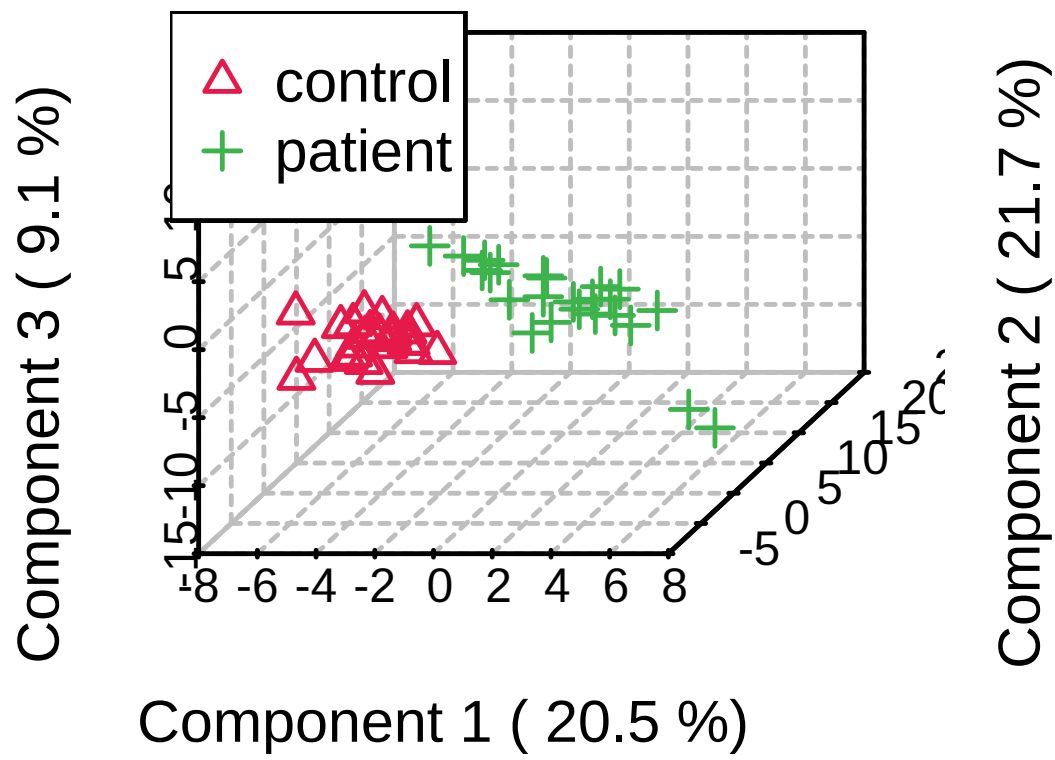


Figure 9: 3D scores plot between the selected PCs. The explained variances are shown in brackets.

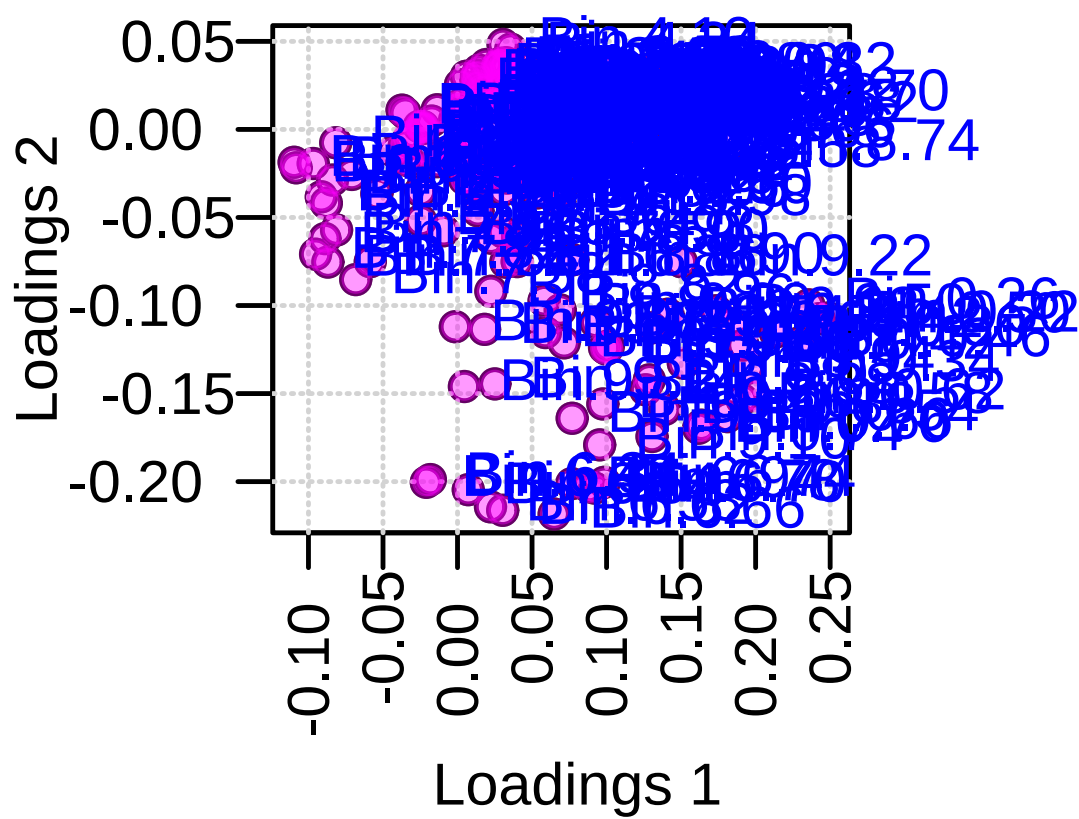


Figure 10: Loadings plot between the selected PCs.

2.4 Hierarchical Clustering

In (agglomerative) hierarchical cluster analysis, each sample begins as a separate cluster and the algorithm proceeds to combine them until all samples belong to one cluster. Two parameters need to be considered when performing hierarchical clustering. The first one is similarity measure - Euclidean distance, Pearson's correlation, Spearman's rank correlation. The other parameter is clustering algorithms, including average linkage (clustering uses the centroids of the observations), complete linkage (clustering uses the farthest pair of observations between the two groups), single linkage (clustering uses the closest pair of observations) and Ward's linkage (clustering to minimize the sum of squares of any two clusters). Heatmap is often presented as a visual aid in addition to the dendrogram.

Hierarchical clustering is performed with the `hclust` function in package `stat`. Figure 14 shows the clustering result in the form of a dendrogram. Figure 15 shows the clustering result in the form of a heatmap.

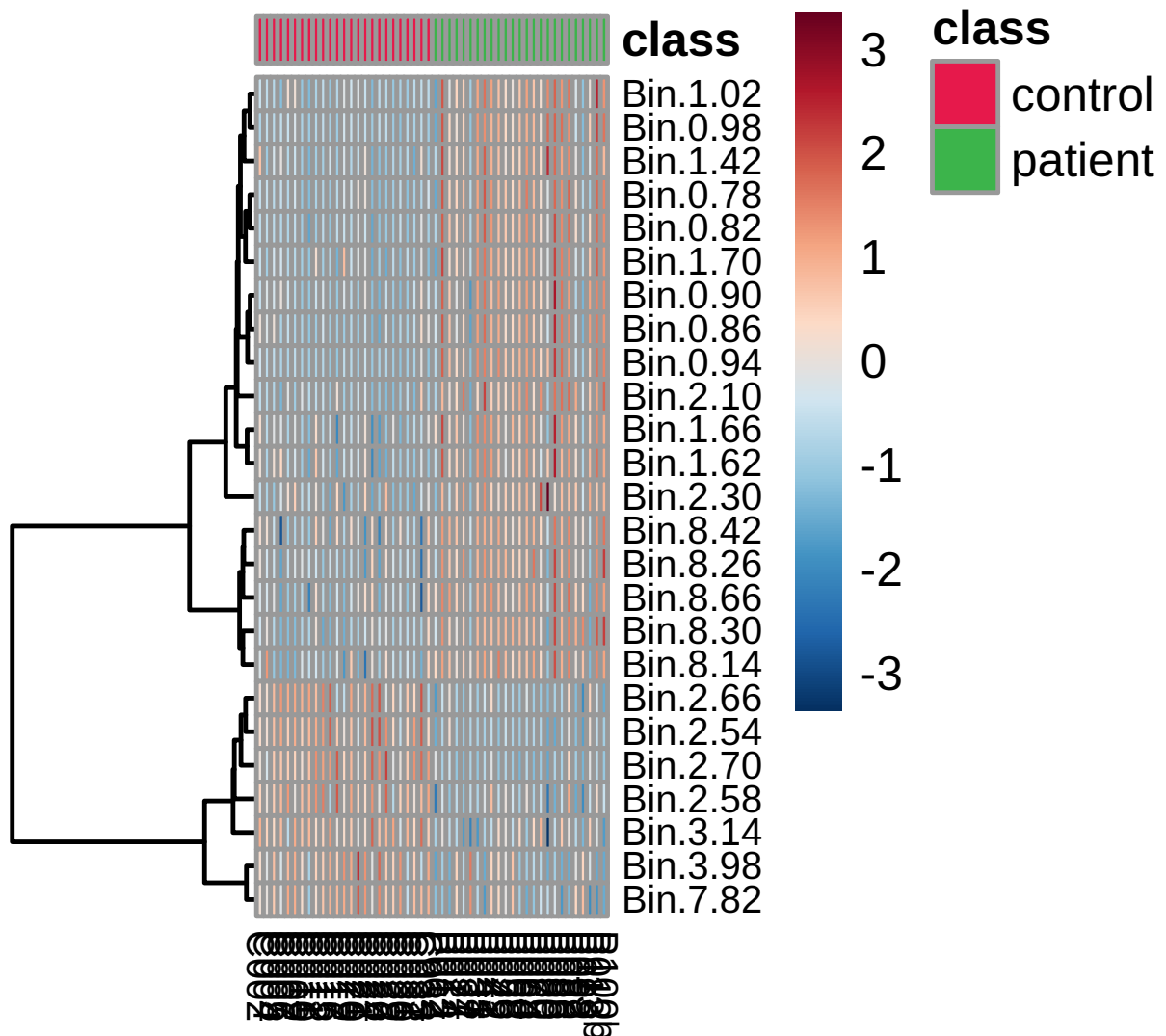


Figure 12: Clustering result shown as heatmap (distance measure using `euclidean`, and clustering algorithm using `ward.D`).

3 Appendix: R Command History

```
[1] "mSet<-InitDataObjects(\"specbin\", \"stat\", FALSE, 150)"
[2] "mSet<-Read.TextData(mSet, \"Replacing_with_your_file_path\", \"rowu\", \"disc\");"
[3] "mSet<-SanityCheckData(mSet)"
[4] "mSet<-PerformSanityClosure (mSet);"
[5] "mSet<-CheckContainsBlank(mSet)"
[6] "mSet<-FilterVariable(mSet, \"F\", 20, \"none\", -1, \"mean\", 0, F,10.0)"
[7] "mSet<-PreparePrenormData(mSet)"
[8] "mSet<-Normalization(mSet, \"NULL\", \"Log2Norm\", \"AutoNorm\", ratio=FALSE, ratioNum=20)"
[9] "mSet<-PlotNormSummary(mSet, \"norm_0_\", \"png\", 150, width=NA)"
[10] "mSet<-PlotSampleNormSummary(mSet, \"snorm_0_\", \"png\", 150, width=NA)"
[11] "mSet<-Normalization(mSet, \"SumNorm\", \"LogNorm\", \"ParetoNorm\", ratio=FALSE, ratioNum=20)"
[12] "mSet<-PlotNormSummary(mSet, \"norm_1_\", \"png\", 150, width=NA)"
[13] "mSet<-PlotSampleNormSummary(mSet, \"snorm_1_\", \"png\", 150, width=NA)"
[14] "mSet<-Ttests.Anal(mSet, F, 0.05, FALSE, TRUE, \"fdr\", FALSE)"
[15] "mSet<-PlotTT(mSet, \"tt_0_\", \"png\", 150, width=NA)"
[16] "mSet<-Ttests.Anal(mSet, F, 0.05, FALSE, FALSE, \"fdr\", FALSE)"
[17] "mSet<-PlotTT(mSet, \"tt_1_\", \"png\", 150, width=NA)"
[18] "mSet<-PCA.Anal(mSet)"
[19] "mSet<-PlotPCAPairSummaryMeta(mSet, \"pca_pair_0_\", \"png\", 96, width=NA, 5, \"NA\", \"NA\")"
[20] "mSet<-PlotPCAScree(mSet, \"pca_screes_0_\", \"png\", 150, width=NA, 5)"
[21] "mSet<-PlotPCA2DScore(mSet, \"pca_score2d_0_\", \"png\", 150, width=NA, 1,2,0.95,0,0, \"na\")"
[22] "mSet<-PlotPCALoading(mSet, \"pca_loading_0_\", \"png\", 150, width=NA, 1,2);"
[23] "mSet<-PlotPCABiplot(mSet, \"pca_biplot_0_\", \"png\", 150, width=NA, 1,2,10)"
[24] "mSet<-PlotPCA3DLoading(mSet, \"pca_loading3d_0_\", \"json\", 1,2,3)"
[25] "mSet<-PlotPCAPairSummaryMeta(mSet, \"pca_pair_1_\", \"png\", 96, width=NA, 5, \"NA\", \"NA\")"
[26] "mSet<-PLSR.Anal(mSet, reg=TRUE)"
[27] "mSet<-PlotPLSPairSummary(mSet, \"pls_pair_0_\", \"png\", 96, width=NA, 5)"
[28] "mSet<-PlotPLS2DScore(mSet, \"pls_score2d_0_\", \"png\", 150, width=NA, 1,2,0.95,0,0, \"na\")"
[29] "mSet<-PlotPLS3DScoreImg(mSet, \"pls_score3d_0_\", \"png\", 150, width=NA, 1,2,3, 40)"
[30] "mSet<-PlotPLSLoading(mSet, \"pls_loading_0_\", \"png\", 150, width=NA, 1, 2);"
[31] "mSet<-PlotPLS3DLoading(mSet, \"pls_loading3d_0_\", \"json\", 1,2,3)"
[32] "mSet<-PlotPLS.Imp(mSet, \"pls_imp_0_\", \"png\", 150, width=NA, \"vip\", \"Comp. 1\", 15,FALSE)"
[33] "mSet<-PlotPLSBiplot(mSet, \"pls_biplot_0_\", \"png\", 150, width=NA, 1,2,10)"
[34] "mSet<-PlotPLSPairSummary(mSet, \"pls_pair_1_\", \"png\", 96, width=NA, 5)"
[35] "mSet<-PlotSubHeatMap(mSet, \"heatmap_1_\", \"png\", 150, width=NA, \"norm\", \"row\", \"euclid)"
[36] "mSet<-SaveTransformedData(mSet)"
[37] "mSet<-PreparePDFReport(mSet, \"guest3216449863702285686\")\n"
```

The report was generated on Wed Apr 29 20:07:46 2026 with R version 4.5.2 (2025-10-31), OS system: Linux.